

Experiment No. 2

Title

Implementation of Uninformed Search Algorithms (Breadth-First Search and Depth-First Search) to Solve a Maze Problem

Aim

To implement **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** algorithms to find a path from the start position to the goal in a maze and compare their performance.

Objectives

1. Understand state-space representation in Artificial Intelligence.
2. Implement uninformed search algorithms.
3. Apply BFS and DFS to solve a maze problem.
4. Compare BFS and DFS in terms of optimality and memory usage.
5. Visualize the path generated by both algorithms.

Problem Statement

Given a maze represented as a two-dimensional grid, where:

- **0** represents a free cell.
- **1** represents an obstacle (wall).
- **S** is the Start state.
- **G** is the Goal state.

Find a path from **S** to **G** using **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**.

Theory

Artificial Intelligence Search

Search is a fundamental problem-solving technique in Artificial Intelligence. The objective is to move from the **initial state** to the **goal state** by exploring possible actions.

A search problem consists of:

- Initial State
- Goal State
- State Space

- Successor Function
 - Goal Test
 - Path Cost
-

Uninformed Search

Uninformed (Blind) Search algorithms have no knowledge about the distance to the goal.

They only use

- Initial State
- Successor Function
- Goal Test

No heuristic information is used.

Examples

- Breadth First Search
 - Depth First Search
 - Uniform Cost Search
-

Maze Representation

```
S 0 1 0 0
1 0 1 0 1
0 0 0 0 0
0 1 1 1 0
0 0 0 G 0
```

where

S = Start

G = Goal

0 = Free Cell

1 = Wall

State Space

Each cell in the maze represents a state.

Possible actions

- Move Up
- Move Down
- Move Left
- Move Right

provided the next cell is inside the maze and is not a wall.

Breadth First Search (BFS)

Principle

Explores the maze **level by level**.

Uses

Queue (FIFO)

BFS always finds the **shortest path** in an unweighted maze.

Algorithm

1. Insert Start node into Queue.
 2. Mark Start as visited.
 3. Remove front node.
 4. If Goal is reached, stop.
 5. Add all valid neighbouring cells.
 6. Repeat until Goal is found.
-

Depth First Search (DFS)

Principle

Explores one path completely before backtracking.

Uses

Stack (or Recursion)

DFS may not return the shortest path.

Algorithm

1. Visit Start node.
 2. Mark visited.
 3. Visit first neighbour recursively.
 4. Continue until Goal is reached.
 5. Backtrack whenever necessary.
-

Python Program

```
from collections import deque

maze = [
    [0,0,1,0,0],
    [1,0,1,0,1],
    [0,0,0,0,0],
    [0,1,1,1,0],
    [0,0,0,0,0]
]

rows = len(maze)
cols = len(maze[0])

start = (0,0)
goal = (4,3)

directions = [(-1,0), (1,0), (0,-1), (0,1)]

# ----- BFS -----

def bfs():

    queue = deque([(start, [start])])

    visited = set()

    while queue:

        (x,y),path = queue.popleft()

        if (x,y)==goal:
            return path

        if (x,y) in visited:
            continue

        visited.add((x,y))

        for dx,dy in directions:

            nx=x+dx
            ny=y+dy
```

```

        if 0<=nx<rows and 0<=ny<cols:
            if maze[nx][ny]==0 and (nx,ny) not in visited:
                queue.append((nx,ny),path+[nx,ny]))

    return None

# ----- DFS -----

def dfs():

    stack=[(start,[start])]

    visited=set()

    while stack:

        (x,y),path=stack.pop()

        if (x,y)==goal:
            return path

        if (x,y) in visited:
            continue

        visited.add((x,y))

        for dx,dy in directions:

            nx=x+dx
            ny=y+dy

            if 0<=nx<rows and 0<=ny<cols:
                if maze[nx][ny]==0 and (nx,ny) not in visited:
                    stack.append((nx,ny),path+[nx,ny]))

    return None

print("BFS Path")
print(bfs())

print("\nDFS Path")
print(dfs())

```

Sample Output

BFS Path

```

[(0,0),
 (0,1),
 (1,1),
 (2,1),
 (2,2),
 (2,3),
 (2,4),
 (3,4),
 (4,4),

```

(4, 3)]

DFS Path

```
[ (0, 0) ,  
  (0, 1) ,  
  (1, 1) ,  
  (2, 1) ,  
  (2, 2) ,  
  (2, 3) ,  
  (2, 4) ,  
  (3, 4) ,  
  (4, 4) ,  
  (4, 3) ]
```

(Note: Depending on the order in which neighbours are explored, DFS may return a different valid path.)

Complexity Analysis

Parameter	BFS	DFS
Search Strategy	Level-wise	Depth-wise
Data Structure	Queue	Stack
Completeness	Yes	Yes (finite maze)
Optimal	Yes	No
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$	$O(V)$

Advantages

BFS

- Finds the shortest path.
- Complete for finite search spaces.
- Suitable for pathfinding in unweighted mazes.

DFS

- Lower memory usage in many cases.
 - Faster when the goal is deep.
 - Simple recursive implementation.
-

Limitations

BFS

- High memory consumption.
- Slower for very large search spaces.

DFS

- May not find the shortest path.
 - Can spend excessive time exploring deep branches.
-

Maze look

```
print(maze[0])
print(maze[1])
print(maze[2])
print(maze[3])
print(maze[4])
```

```
[0, 0, 0, 0, 0]
[0, 1, 1, 0, 1]
[0, 0, 0, 0, 0]
[0, 1, 1, 0, 1]
[0, 1, 0, 0, 0]
```

Code

```
from collections import deque

maze = [
    [0,0,0,0,0],
    [0,1,1,0,1],
    [0,0,0,0,0],
    [0,1,1,0,1],
    [0,1,0,0,0]
]

rows = len(maze)
cols = len(maze[0])

start = (1,1)
goal = (4,4)

directions = [(-1,0), (1,0), (0,-1), (0,1)]
def bfs():
```

```

queue = deque([(start, [start])])
visited = set()
while queue:
    (x,y),path = queue.popleft()
    if (x,y)==goal:
        return path
    if (x,y) in visited:
        continue
    visited.add((x,y))
    for dx,dy in directions:
        nx=x+dx
        ny=y+dy
        if 0<=nx<rows and 0<=ny<cols:
            if maze[nx][ny]==0 and (nx,ny) not in visited:
                queue.append(((nx,ny),path+[(nx,ny)]))
return None

def dfs():
    stack=[(start, [start])]
    visited=set()
    while stack:
        (x,y),path=stack.pop()
        if (x,y)==goal:
            return path
        if (x,y) in visited:
            continue
        visited.add((x,y))
        for dx,dy in directions:
            nx=x+dx
            ny=y+dy
            if 0<=nx<rows and 0<=ny<cols:
                if maze[nx][ny]==0 and (nx,ny) not in visited:
                    stack.append(((nx,ny),path+[(nx,ny)]))
    return None

print("BFS Path")
print(bfs())
print("\nDFS Path")
print(dfs())

```

Output

BFS Path

```
[(1, 1), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (4, 4)]
```

DFS Path

```
[(1, 1), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (4, 4)]
```

```

BFS Path
[(1, 1), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (4, 4)]

DFS Path
[(1, 1), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (4, 4)]

]: print(maze[0])
   print(maze[1])
   print(maze[2])
   print(maze[3])
   print(maze[4])

[0, 0, 0, 0, 0]
[0, 1, 1, 0, 1]
[0, 0, 0, 0, 0]
[0, 1, 1, 0, 1]
[0, 1, 0, 0, 0]

```


Applications

- Robot navigation
 - Autonomous vehicle path planning
 - Game AI
 - Maze solving
 - GPS navigation
 - Network routing
 - Puzzle solving
 - AI planning
-

Viva Questions

1. What is a state-space search problem?
 2. What is an uninformed search algorithm?
 3. Why is BFS guaranteed to find the shortest path in an unweighted maze?
 4. Which data structure is used by BFS?
 5. Which data structure is used by DFS?
 6. Why can DFS return a non-optimal path?
 7. What are the successor states in a maze problem?
 8. What is the time complexity of BFS and DFS?
 9. What is the role of the visited set?
 10. List two AI applications where BFS and DFS are used.
-

Result

The Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms were successfully implemented to solve a maze navigation problem. The algorithms explored the maze as a state-space search, identifying a valid path from the start state to the goal state. BFS successfully produced the shortest path due to its level-wise exploration strategy, while DFS found a valid path by exploring deeper branches first. The experiment demonstrated the working principles, performance characteristics, and practical applications of uninformed search techniques in Artificial Intelligence.